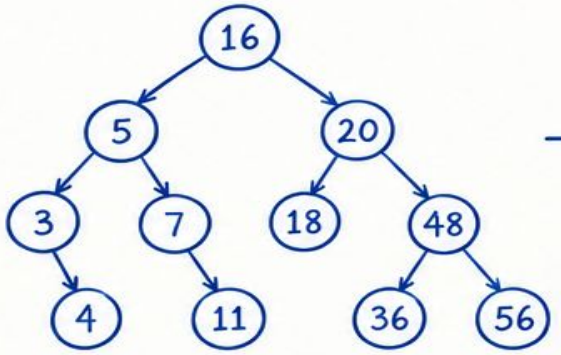
Binary Search Tree Operations

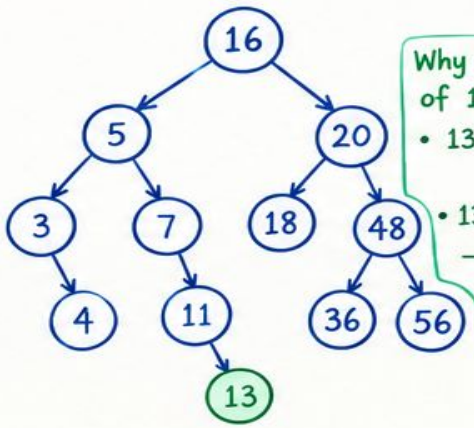
Sorting ,Search, Insert and Delete in Binary Search Tree

1) Build BST (Insert in order)



BST after inserting all numbers except 13 (and before search & delete).

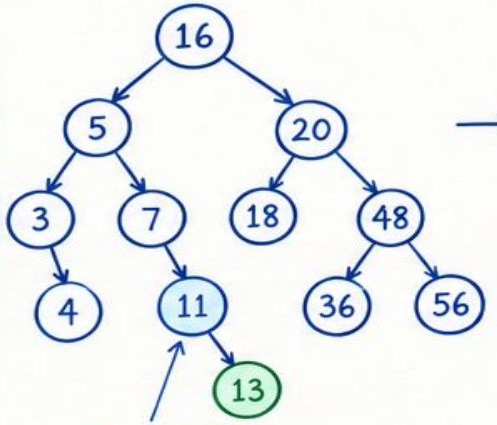
2) Insert 13 (e.g. 13)



Why 13 is the right child of 11 ?

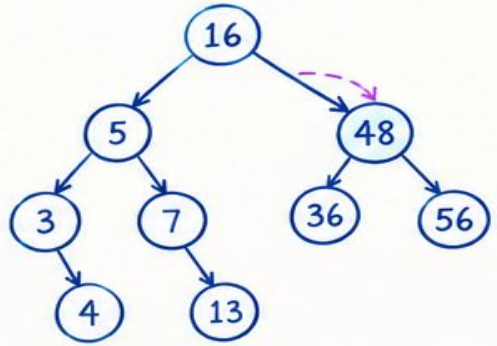
- 13 > 11 → go to right of 11.
- 13 < 16, 13 < 20, 13 > 11 → so it becomes the right child of 11.

3) Search for 11



Found !

4) Delete 20



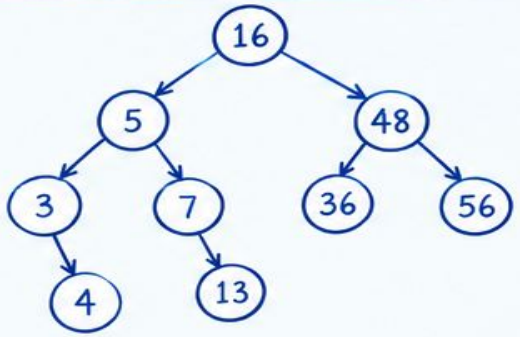
Why 48 replaces 20?

- 20 has two children (18 and 48).
- Use the in-order successor (smallest in right subtree).
- Smallest in right subtree of 20 is 48 (no left child).
- Replace 20 with 48 and remove 48 from its original position.

Summary of Operations

- **Sorting** : In-order traversal gives sorted order.
- **Search** : Compare and move left/right (e.g. 11 found).
- **Insert** : Place at correct position (e.g. 13 → right of 11).
- **Delete** : If node has two children, replace with in-order successor (e.g. 20 → 48).

Final BST after all operations



Final tree (after inserting 13 and deleting 20).

Key Points

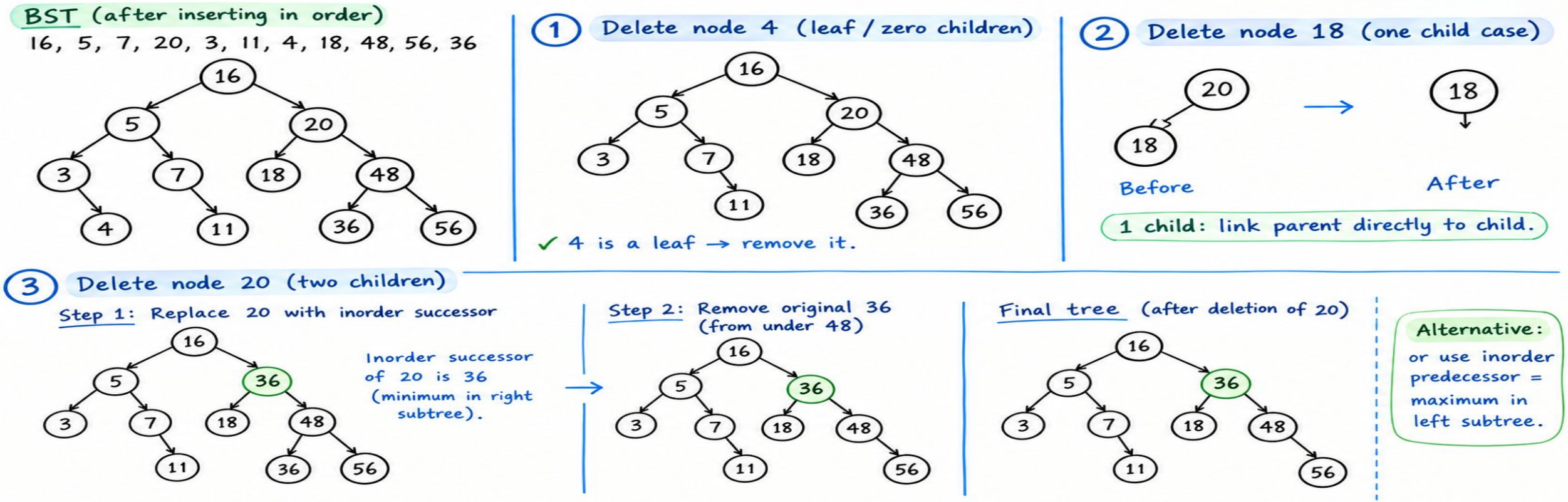
- 13 is right child of 11 because 13 > 11 and it follows the BST rule (greater values go right).
- Deleting 20: since it had two children, we replaced it with its in-order successor 48 (the smallest node in the right subtree) and removed 48 from its original place.
- The BST property is maintained throughout.

Binary Search Tree Operations

Delete in Binary Search Tree

- **For Child 0**
- **For Child 1**
- **For Child 2**
  - **Replace the delete node with (inorder predecessor)**
  - **Replace the delete node with (Inorder successor)**

16, 5, 7, 20, 3, 11, 4, 18, 48, 56, 36



Binary Search Tree Operations

The time complexity of binary search is

1. Best Case- **O(1) i.e. constant**
2. Average Case- **O(log n)**
3. Worst Case- ***O(log n)***